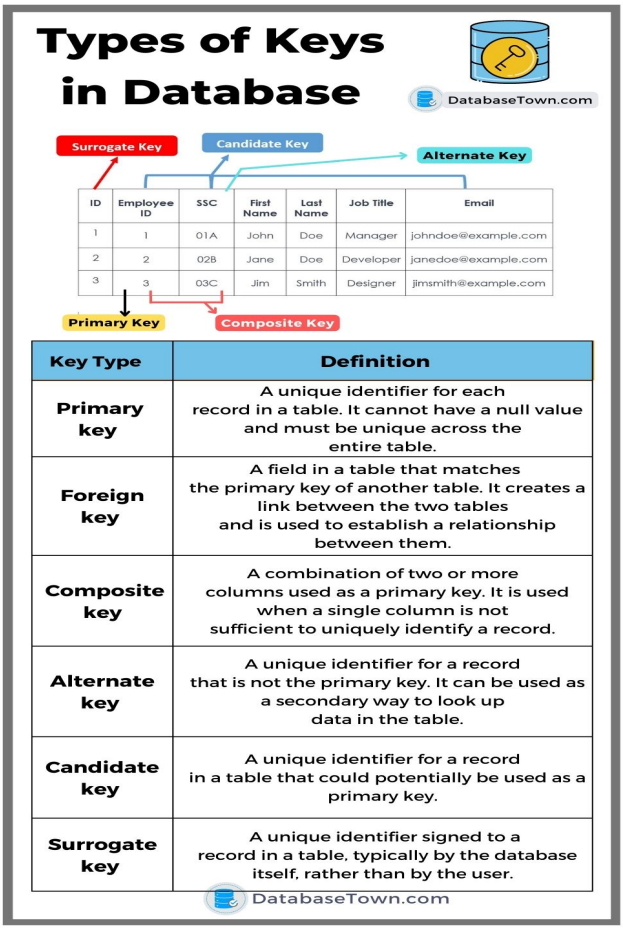
Day3- SQL

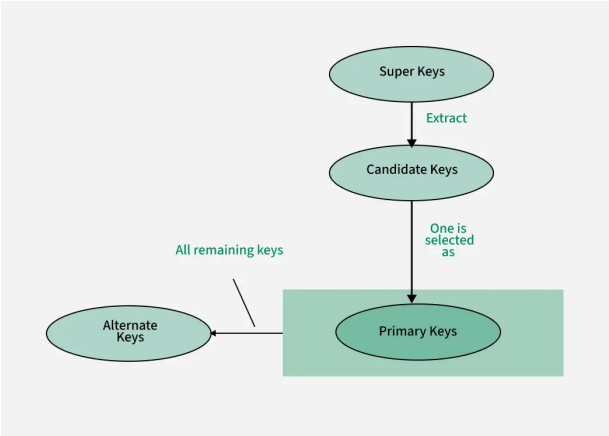
Sunday, 5 April 2026 9:04 AM

Keys in databases:

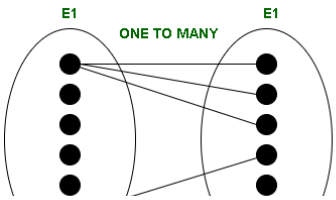


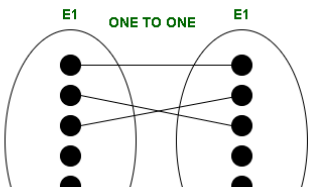
ID	Employee ID	SSC	First Name	Last Name	Job Title	Email
1	1	01A	John	Doe	Manager	johndoe@example.com
2	2	02B	Jane	Doe	Developer	janedoe@example.com
3	3	03C	Jim	Smith	Designer	jimsmith@example.com

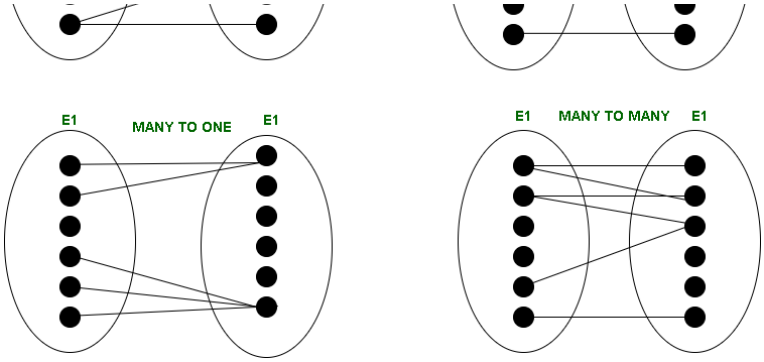
Key Type	Definition
Primary key	A unique identifier for each record in a table. It cannot have a null value and must be unique across the entire table.
Foreign key	A field in a table that matches the primary key of another table. It creates a link between the two tables and is used to establish a relationship between them.
Composite key	A combination of two or more columns used as a primary key. It is used when a single column is not sufficient to uniquely identify a record.
Alternate key	A unique identifier for a record that is not the primary key. It can be used as a secondary way to look up data in the table.
Candidate key	A unique identifier for a record in a table that could potentially be used as a primary key.
Surrogate key	A unique identifier signed to a record in a table, typically by the database itself, rather than by the user.



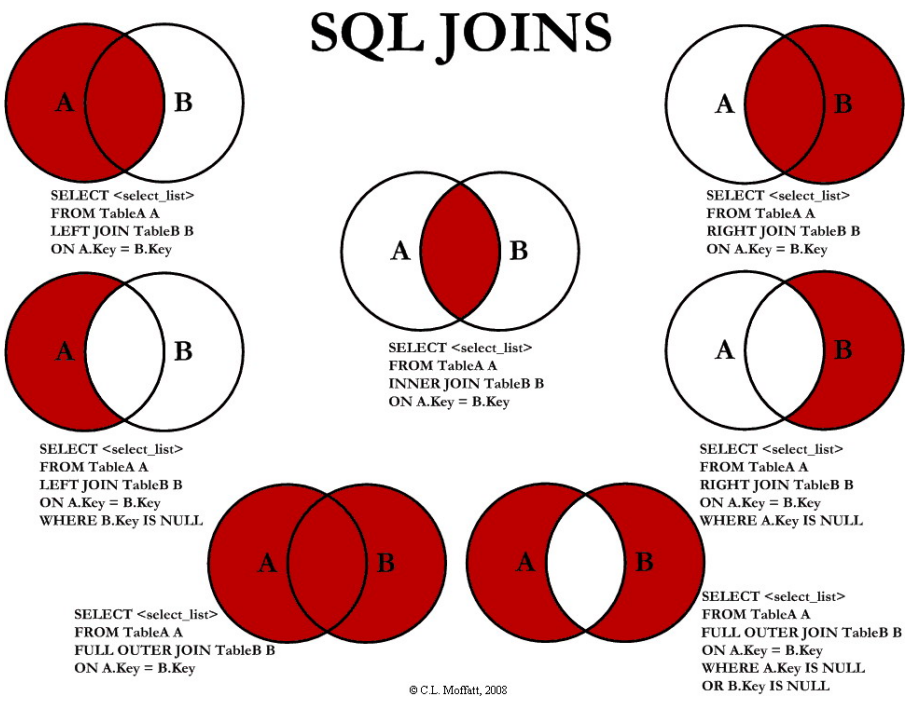
Cardinality in Databases:

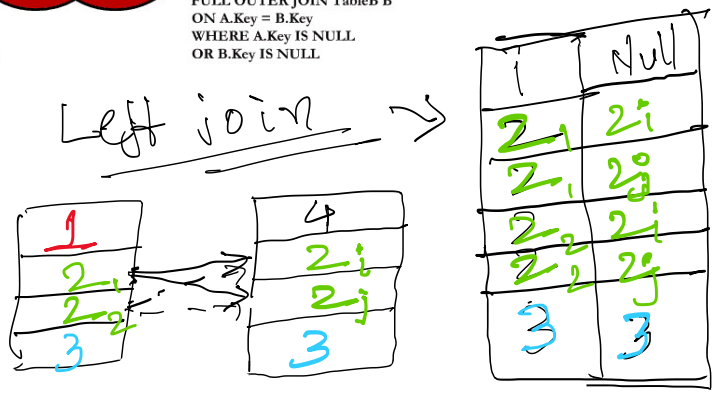






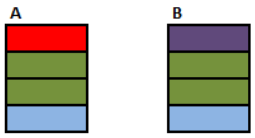
Joins in SQL:





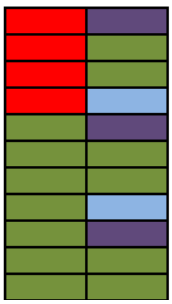
Tabular representation

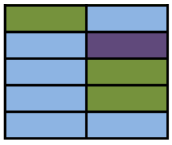
Tables



SELECT A.Colour, B.Colour FROM A CROSS JOIN B

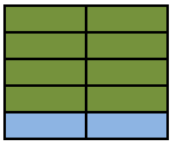
The cross join (or cartesian product) produces a result with every combination of the rows from the two tables. Each table has 4 rows so this produces 16 rows in the result.





SELECT A.Colour, B.Colour FROM A INNER JOIN B ON A.Colour = B.Colour

The inner join logically returns all rows from the cross join that match the join condition. In this case five do.



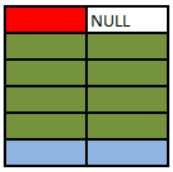
SELECT A.Colour, B.Colour FROM A INNER JOIN B ON A.Colour NOT IN ('Green','Blue')

The inner join condition need not necessarily be an equality condition and it need not reference columns from both (or even either) of the tables. Evaluating A.Colour NOT IN ('Green','Blue') on each row of the cross join returns.

An inner join condition of 1=1 would evaluate to true for every row in the cross join so the two are equivalent

SELECT A.Colour, B.Colour FROM A LEFT OUTER JOIN B ON A.Colour = B.Colour

Outer Joins are logically evaluated in the same way as inner joins except that if a row from the left table (for a left join) does not join with any rows from the right hand table at all it is preserved in the result with NULL values for the right hand columns.



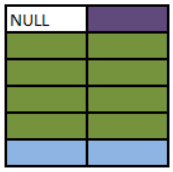
SELECT A.Colour, B.Colour FROM A LEFT OUTER JOIN B ON A.Colour = B.Colour WHERE B.Colour IS NULL

This simply restricts the previous result to only return the rows where B.Colour IS NULL. In this particular case these will be the rows that were preserved as they had no match in the right hand table and the query returns the single red row not matched in table B. This is known as an anti semi join.

It is important to select a column for the IS NULL test that is either not nullable or for which the join condition ensures that any NULL values will be excluded in order for this pattern to work correctly and avoid just bringing back rows which happen to have a NULL value for that column in addition to the un matched rows.

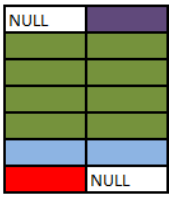
SELECT A.Colour, B.Colour FROM A RIGHT OUTER JOIN B ON A.Colour = B.Colour

Right outer joins act similarly to left outer joins except they preserve non matching rows from the right table and null extend the left hand columns.

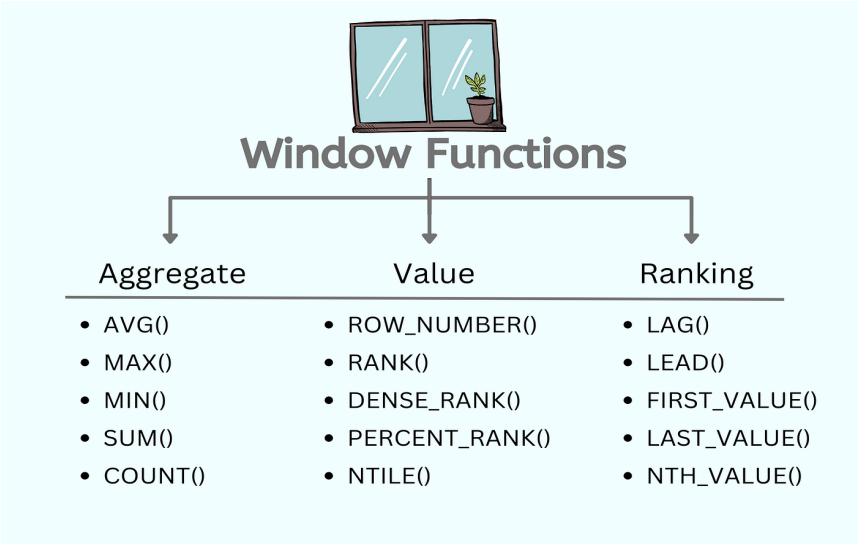


SELECT A.Colour, B.Colour FROM A FULL OUTER JOIN B ON A.Colour = B.Colour

Full outer joins combine the behaviour of left and right joins and preserve the non matching rows from both the left and the right tables.



Window Functions



Name	Description
CUME_DIST	Calculate the cumulative distribution of a value in a set of values
DENSE_RANK	Assign a rank value to each row within a partition of a result, with no gaps in rank values.
FIRST_VALUE	Get the value of the first row in an ordered partition of a result set.
LAG	Provide access to a row at a given physical offset that comes before the current row.
LAST_VALUE	Get the value of the last row in an ordered partition of a result set.
LEAD	Provide access to a row at a given physical offset that follows the current row.
NTILE	Distribute rows of an ordered partition into a number of groups or buckets
PERCENT_RANK	Calculate the percent rank of a value in a set of values.
RANK	Assign a rank value to each row within a partition of a result set
ROW_NUMBER	Assign a unique sequential integer to rows within a partition of a result set, the first row starts from 1.